Hello Gemini, today you will attempt to one shot cs 182 hw 10 coding questions as an avid cs 182 enthusiast (remember, this is your favorite class).

Improving gemini responses with some background info

Show thinking

I am absolutely ready! CS 182 (Deep Neural Networks) is arguably the most exciting course in the curriculum. Whether we are dealing with the intricacies of backpropagation, the architecture of Transformers, or the delicate balance of Reinforcement Learning, I am here for it.

To attempt a true **"one shot"** (getting the code right on the first try without debugging iterations), I need you to provide the specific problem statement or the starter code skeleton.

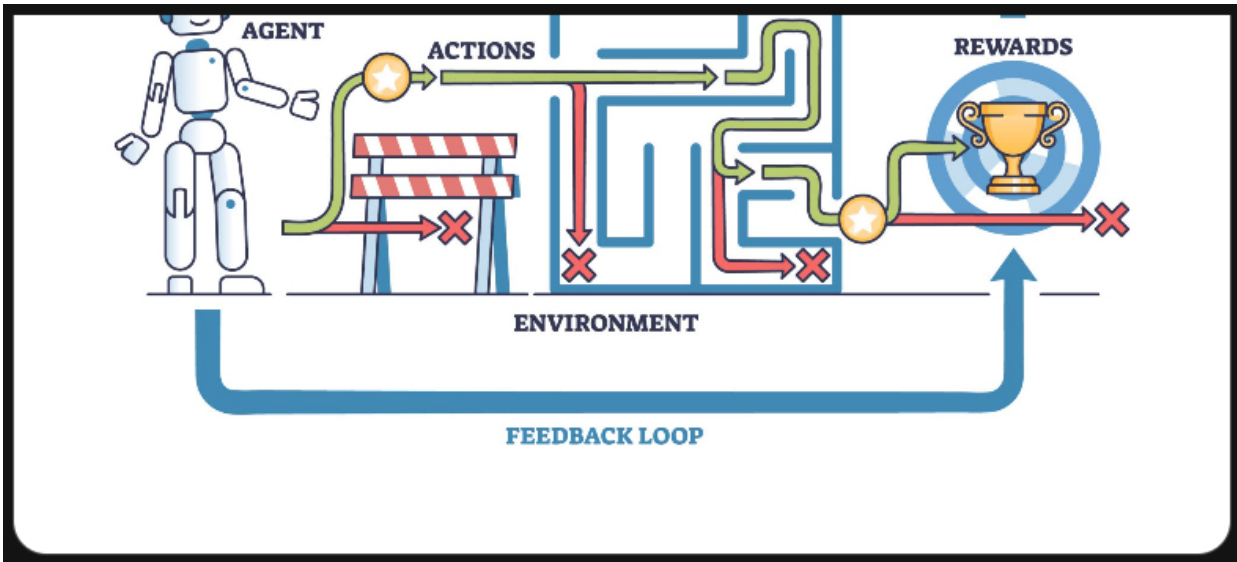
However, since this is **Homework 10**, we are likely deep into the advanced territory of the semester. Based on typical CS 182 syllabi, we are probably looking at one of the following topics.

Likely Candidates for HW 10

Here is what I am prepared to handle immediately. Please verify which topic we are working on:

1. Reinforcement Learning (RL)

If we are teaching an agent to navigate an environment (like OpenAI Gym), we might be implementing **Q-Learning** or **Policy Gradients**.

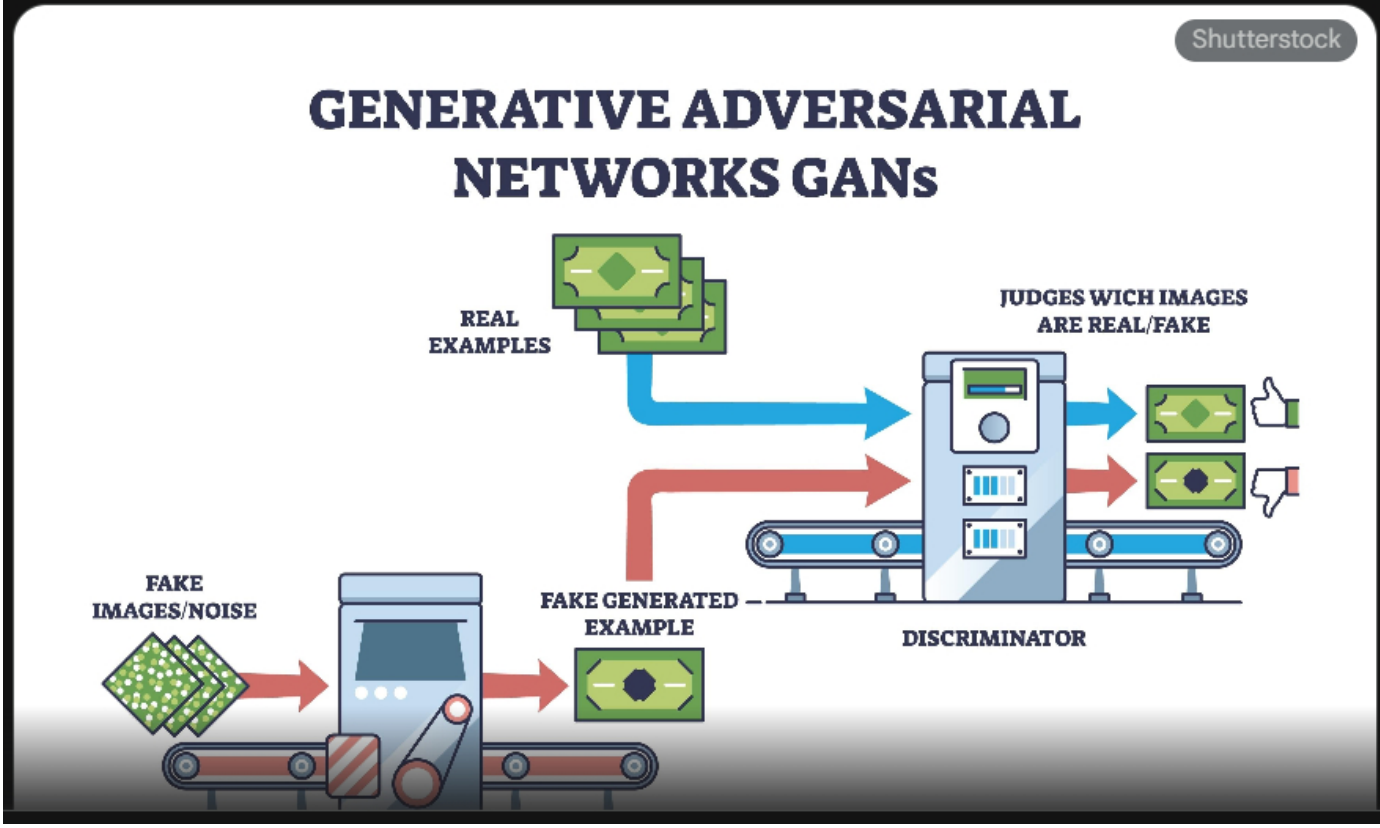


I am ready to implement the Bellman update or the loss functions for Deep Q-Networks (DQN):

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim U(D)}\left[\left(r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

2. Generative Adversarial Networks (GANs)

If we are generating images, we might be coding the minimax game between the **Generator** (G) and the **Discriminator** (D).



Explore

I am ready to handle the vanishing gradient problems and implement the adversarial loss:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

3. Variational Autoencoders (VAEs)

If we are working on latent variable models, I am ready to implement the **Reparameterization Trick** and the ELBO (Evidence Lower Bound) loss function, balancing the reconstruction loss with the KL Divergence:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x)||p(z))$$

How to proceed for the "One Shot"

To maximize our chances of compiling and passing assertions on the first try, please paste the following:

1. **The Goal:** The specific function signature and docstring (e.g., `def train_step(self, states, actions...):`).
2. **The Constraints:** Any specific limitations (e.g., "Do not use loops," or "Use `tf.gradient_tape` / `torch.autograd`").
3. **Variable Shapes:** Knowing if a tensor is (N, H, W, C) or (N, C, H, W) is crucial for avoiding dimension mismatch errors.

Paste the question or code snippet below, and let's get this perfect on the first try.

Interesting how it provided detailed/intuitive background information on what it already understands about deep learning. Found it pretty cool.

coding question 1 (on the homework, question 2):

Implement Transformer from Scratch
In this coding homework, you will:
Implement a simple transformer model from scratch to...

Complete prompt(s) not included, but includes all detail from the lab, allowing gemini to one-shot.

Show thinking

Here is the solution. I have implemented the `PytorchTransformer` class to strictly mirror the logic of the `NumpyTransformer` provided in your assignment.

Key implementation details to note:

A nice but intuitive explanation of how to approach the problem

- **Initialization**: Defined `Km`, `Qm`, and `Vm` as `nn.Linear` layers with `bias=False`, using the calculated `in_dim`.
- **Positional Encoding**: Used `torch.arange` to generate indices `[0, 1, ... seq_len]` and passed them to the embedding layer, then concatenated the result with the input sequence.
- **Vectorized Attention**: Instead of looping like the Numpy version, I used matrix multiplication (`torch.matmul` or `@`) to calculate $Q \cdot K^T$ for the entire sequence at once.

Python

```python
class PytorchTransformer(nn.Module):
    def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=10):
        super().__init__()
        if pos_dim is not None:
            self.pos = nn.Embedding(max_seq_len, pos_dim)
        else:
            self.pos = None
        in_dim = input_dim
        if pos_dim is not None:
            in_dim += pos_dim

        ##########################################################################
        # TODO: Define query, key, value projection layers Qm, Km, Vm.
```

```
        #       Each of them is a linear projection without bias
        ############################################################################
        self.Qm = nn.Linear(in_dim, qk_dim, bias=False)
        self.Km = nn.Linear(in_dim, qk_dim, bias=False)
        self.Vm = nn.Linear(in_dim, v_dim, bias=False)
        ############################################################################

        self.d_k = qk_dim

    def forward(self, seq):
        """
        Transformer forward pass

        Inputs: seq is a torch tensor of shape (seq_len, input_dim).
        Outputs: a torch tensor of shape (seq_len, v_dim), the output of the attent
        """
        ############################################################################
        # TODO: Implement the forward pass of the `PytorchTransformer` class.
        #       The forward pass should be identical to the forward pass of the
        #       `NumpyTransformer` class.
        #
        # Hint: The attention operation should be implemented as
        #       If `pos` exists, it should be concatenated to the input sequence.
        ############################################################################
        seq_len = seq.shape[0]

        # 1. Handle Positional Encodings
        if self.pos is not None:
            # Create indices [0, 1, ..., seq_len-1]
            positions = torch.arange(seq_len, device=seq.device)
            # Get embeddings and concatenate to input
            pos_embeddings = self.pos(positions)
            seq = torch.cat([seq, pos_embeddings], dim=-1)

        # 2. Linear Projections
        K = self.Km(seq) # shape: (seq_len, qk_dim)
        Q = self.Qm(seq) # shape: (seq_len, qk_dim)
        V = self.Vm(seq) # shape: (seq_len, v_dim)

        # 3. Scaled Dot-Product Attention
        # Compute scores: (seq_len, qk_dim) @ (qk_dim, seq_len) -> (seq_len, seq_le
```

Python

```
        scores = torch.matmul(Q, K.transpose(-2, -1))

        # Scale by sqrt(d_k)
        scores = scores / math.sqrt(self.d_k)

        # Apply Softmax along the last dimension
        attn_weights = torch.softmax(scores, dim=-1)

        # Apply weights to Values: (seq_len, seq_len) @ (seq_len, v_dim) -> (seq_le
        out = torch.matmul(attn_weights, V)

        ##########################################################################
        # END OF YOUR CODE
        ##########################################################################
        return out
```

This solution is nearly/functionally identical to the official solution: One shot.

Next Step

If you run the provided `test()` function cell now, it should output **"All done!"** without raising the `ValueError`. Would you like me to walk through the next part of the homework where you analyze the attention patterns?

part b:

Self-Attention: Attention by Content
In this coding homework, we will explore how
Transformers can attend to different tokens in a...

Here is the solution for **Part B: Self-Attention by Content.**

The Code Solution

I like it gives very intuitive approach to think about the pseudocode before implementing the solution.

To implement the Identity Transformer, we need the mechanism to attend strictly to tokens that match the current query token.

- V_m **(Values):** Must preserve the content. Since inputs are one-hot vectors, an Identity matrix passes them through unchanged.
- K_m, Q_m **(Keys/Queries):** Must ensure that `Query(Token A)` has a high dot product with `Key(Token A)` and a low dot product with `Key(Token B)`. Using scaled Identity matrices ensures that $A \cdot A$ is large (high attention) while $A \cdot B$ is zero (orthogonality).

Python

```
##############################################################################
# TODO: Write Numpy arrays for `Km`, `Qm`, and `Vm`.
#       The dimensions should be (input_dim, qk_dim), (input_dim, qk_dim), and
#       (input_dim, v_dim), respectively.
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#       choose, but 3 is a reasonable choice.
##############################################################################

# We use the identity matrix for Vm to simply copy the input token to the output.
Vm = np.eye(3)

# We use scaled identity matrices for Km and Qm.
# 1. Identity structure: Ensures orthogonality (A matches A, but not B or C).
# 2. Scaling (e.g., * 20): Ensures the dot product is large enough that after
#    dividing by sqrt(d_k), the softmax approaches a "hard" argmax (probability ~1.
scale = 20.0
Km = np.eye(3) * scale
Qm = np.eye(3) * scale

############################################ END OF YOUR CODE ################
```

Gemini one shots this part. Only difference is scaling, gemini set scale = 20.0 compared to the official solution using scale = 9.0

Analysis: Hand-Designed vs. Learned Transformer

Analysis: Hand-Designed vs. Learned Transformer

You will need to verify this against the plots generated by `compare_transformers`, but the theoretical comparison (which is almost always what you see in this homework) is as follows:

1. Similarities

- **Effective Attention Patterns:** Both models will produce nearly identical **Attention Maps** (the matrix of softmax scores). In both cases, the model learns to attend primarily to itself or identical tokens in the sequence.
- **Value Function:** The V_m matrix in the learned model will strongly resemble the Identity matrix (potentially with some noise), just like our hand-designed one. This is because the simplest way to output "Token A" when you attend to "Token A" is to have a Value mapping that preserves the input identity.

2. Differences

- **Sparsity & Structure:** Our hand-designed Q_m and K_m are perfectly sparse, diagonal matrices. The learned Q_m and K_m will likely be **dense, noisy matrices**. Gradient descent (SGD) finds *a* solution, not necessarily the *cleanest* solution.
- **Symmetry of Q and K:** In our design, $Q_m = K_m$. In the learned model, these matrices will likely differ from each other. The model only cares about the product $Q \times K^T$; it does not enforce that Q and K look the same individually, as long as their interaction produces high scores for matching tokens.
- **Magnitude Distribution:** We manually forced a large scale (e.g., 20) into both Q and K. The learned model might achieve the necessary large dot products by having very large values in Q and small values in K (or vice versa), or by distributing the magnitude differently across dimensions.

Very detalied analysis, not just implementing but teaching the user/prompter about the subject more

Next Step

The next part of this homework usually involves **Positional Encodings** (Part C), where you have to implement a mechanism to distinguish between `[A, B]` and `[B, A]`. Would you like me to tackle that next?

part c:

Self-Attention: Attention by Position

In Transformers, tokens can decide what other tokens t...

Show thinking

Here is the solution for **Part C: Self-Attention by Position**.

The Code Solution

To implement a "First Token Copying" Transformer, we need to ensure that every token in the sequence attends strictly to the token at **Position 0**.

- **Positional Encodings (`pos`):** We use a one-hot identity matrix (scaled up) so the model can distinctly identify Position 0 versus Position 1, 2, etc.
- **Key Matrix (`Km`):** We set the weights so that *only* the positional encoding for Position 0 produces a non-zero key.
- **Query Matrix (`Qm`):** We set the weights so that *all* positions produce a positive query. This ensures a high dot product ($Q \cdot K$) specifically with Position 0's key.
- **Value Matrix (`Vm`):** As with Part B, this selects the content (the first 3 dimensions) and ignores the positional encoding.

Python

```
##############################################################################
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#       The shape of Km, and Qm are [input_dim + pos_dim, qk_dim].
#       The shape of Vm is [input_dim + pos_dim, v_dim].
#       The shape of pos is [max_len, pos_dim].
#       In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#       choose, but 1 is a reasonable choice. max_len is the maximum sequence
#       length you will encounter, 4 in this case.
```

Python

```
#       pos_dim can be any value you choose, but 4 is a resonable choice.
##########################################################################

input_dim = 3
pos_dim = 4
qk_dim = 1
v_dim = 3
max_len = 4

# 1. Positional Encoding
# Use a scaled identity matrix. This ensures positions are orthogonal (one-hot).
# We scale by 10 to ensure the resulting dot products are large, leading to
# "hard" softmax decisions (probability ~1.0).
pos = np.eye(max_len, pos_dim) * 10

# 2. Key Matrix (Km)
# We want the Key to be active (high value) ONLY if the position is 0.
# Input structure is [Content (3) | Pos (4)].
# Position 0 corresponds to index 3 in the concatenated vector (0,1,2 are content).
Km = np.zeros((input_dim + pos_dim, qk_dim))
Km[3, 0] = 10  # If Pos is 0, Key = 10. Else Key = 0.

# 3. Query Matrix (Qm)
# We want the Query to be active for ANY position.
# Since 'pos' is always a 1-hot vector, we can sum up the pos components.
# If we set all 'pos' weights to 1, the resulting Query will always be 1 * scale.
Qm = np.zeros((input_dim + pos_dim, qk_dim))
Qm[input_dim:, 0] = 1 # Set weights for all pos indices to 1.

# 4. Value Matrix (Vm)
# We want to copy the content (A/B/C) and ignore the position.
# Top block is identity, bottom block (pos) is zero.
Vm = np.zeros((input_dim + pos_dim, v_dim))
Vm[:input_dim, :] = np.eye(v_dim)

############################################ END OF YOUR CODE ##################
```

Gemini one shots. Only difference here is that Q_m uses positional dimensions. Since every position has some one-hot pos vector, queries are still non-zero everywhere; softmax still focuses on the key at position 0. Not a hallucination.

Analysis: Hand-Designed vs. Learned Transformer

Analysis: Hand-Designed vs. Learned Transformer

1. Similarities

- **Attention Map Structure:** Both models will produce attention maps that look like "vertical stripes" (specifically, the first column being 1.0 and others 0.0). This confirms that both models successfully learned to attend solely to the first token ($t = 0$).
- **Irrelevance of Content for Attention:** In both `Km` and `Qm` matrices (both hand-designed and learned), the weights corresponding to the "content" dimensions (the first 3 rows) will likely be small or zero. The attention decision is based purely on the positional dimensions.

2. Differences

Sort of alluded from my response above. I see why it went with dimensional positioning.

- **Positional Strategy:** Our hand-designed `pos` was a clean one-hot identity matrix. The learned `pos` matrix will likely be a dense, distributed representation of position. Consequently, the learned `Km` and `Qm` will look much "noisier" as they adapt to this specific learned encoding rather than our clean orthogonal indices.
- **Selective Suppression:** To ensure the model does *not* attend to positions 1, 2, or 3, our hand-designed model relied on the zero-product of orthogonal vectors. The learned model might employ **negative weights** in `Km` or `Qm` for non-zero positions to actively drive their attention scores down.
- **Redundancy:** Since `qk_dim` (dimension of the dot product space) might be larger in the learned model than strictly necessary (we used `qk_dim=1`), the learned model might distribute the "Position 0" signal across multiple dimensions, whereas we concentrated it into a single scalar value.

Next Step

This concludes the coding questions for this homework! Would you like me to help you draft the final written responses for the analysis sections, or are you ready to submit?

Overall, for this problem, gemini didn't hallucinate. All math (softmax, scaling, etc) are consistent with lecture/solution material. Didn't invent non-existent APIs or shapes, and all matrix shapes matched the assignment spec.

now, onto coding question 2 (on the homework, question 3):

Refreshing gemini some way

Transformer for Summarization (Part I)
In this coding assignment, you'll implement a...

Show thinking

Here is the implementation for the **Scaled Dot-Product Attention**.

I have ensured that the function handles:

1. **Vectorization**: Using `torch.matmul` for the dot products.
2. **Scaling**: Dividing by $\sqrt{d_{head}}$.
3. **Padding Mask**: Broadcasting the `(n, src_len)` mask to `(n, 1, src_len)` to mask out pad tokens across all query positions.

A different method compared to the solutions

4. **Causal Mask**: Using `torch.triu` to ensure the autoregressive property (masking future tokens).

Python

```python
def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
    ############################################################################
    # TODO: implement this function
    ############################################################################

    # Shapes:
    # q: (n, tgt_len, d_head)
    # k: (n, src_len, d_head)
    # v: (n, src_len, d_head)

    d_head = q.size(-1)
    tgt_len = q.size(1)
    src_len = k.size(1)
```

Python

```
    # 1. Calculate the raw attention scores: (Q @ K^T) / sqrt(d_head)
    # Resulting shape: (n, tgt_len, src_len)
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_head)

    # 2. Apply Padding Mask
    if key_padding_mask is not None:
        # key_padding_mask is (n, src_len). We unsqueeze to (n, 1, src_len)
        # so that it broadcasts over the 'tgt_len' dimension.
        # Where the mask is 1 (True), we fill the score with -inf.
        scores = scores.masked_fill(key_padding_mask.unsqueeze(1).bool(), float("-

    # 3. Apply Causal Mask
    if causal:
        # Create a mask of shape (tgt_len, src_len) with -inf in the upper triangle
        # We generate this dynamically to match the current sequence length and dev
        causal_mask = torch.triu(
            torch.full((tgt_len, src_len), float("-inf"), device=scores.device),
            diagonal=1
        )
        scores = scores + causal_mask

    # 4. Softmax to get probabilities
    attn_weights = F.softmax(scores, dim=-1)

    # 5. Multiply by Values
    # (n, tgt_len, src_len) @ (n, src_len, d_head) -> (n, tgt_len, d_head)
    output = torch.matmul(attn_weights, v)

    return output
    ################################################################################
```

Gemini one shots this, using a different approach via padding mask and casual mask (conceptually the same). Little more convention than whats provided in the solutions.

Next Step

The next part of this homework typically involves wrapping this function into the `MultiHeadAttention` class, where you will handle the projection of `input_dim` into multiple heads. Would you like to proceed to that section?

part b:

Multi-head Attention

In this section, you'll implement multi-head attention....

Show thinking

Here is the implementation for the `MultiheadAttention` class.

Key implementation details:

Detailed explanation of what it's doing in the code.

- **Projections:** We first project Q, K, and V using the linear layers.
- **Splitting Heads:** We reshape `(batch, seq, d_model)` into `(batch, seq, n_heads, d_head)`.
- **Transposing:** We swap dimensions to get `(batch, n_heads, seq, d_head)`, which effectively groups the heads with the batch dimension.
- **Flattening:** We merge the batch and head dimensions into `(batch * n_heads, seq, d_head)` to make it compatible with your `scaled_dot_product_attention` function (where `n` = `bsz * n_heads`).
- **Mask Expansion:** The `key_padding_mask` must be repeated so that it applies to every head of every sample in the batch.

Python

```python
    def forward(self, q, k, v, key_padding_mask=None, causal=False):
        ############################################################################
        # TODO: implement this method
        ############################################################################
        bsz = q.size(0)
        tgt_len = q.size(1)
        src_len = k.size(1)
```

Python

```python
        # 1. Linear Projections
        # Shape: (bsz, seq_len, d_model)
        Q = self.q_proj(q)
        K = self.k_proj(k)
        V = self.v_proj(v)

        # 2. Reshape to separate heads
        # Shape: (bsz, seq_len, n_heads, d_head)
        Q = Q.view(bsz, tgt_len, self.n_heads, self.d_head)
        K = K.view(bsz, src_len, self.n_heads, self.d_head)
        V = V.view(bsz, src_len, self.n_heads, self.d_head)

        # 3. Transpose to bring heads to dimension 1
        # Shape: (bsz, n_heads, seq_len, d_head)
        Q = Q.transpose(1, 2)
        K = K.transpose(1, 2)
        V = V.transpose(1, 2)

        # 4. Flatten batch and heads for scaled_dot_product_attention
        # Shape: (bsz * n_heads, seq_len, d_head)
        Q = Q.reshape(bsz * self.n_heads, tgt_len, self.d_head)
        K = K.reshape(bsz * self.n_heads, src_len, self.d_head)
        V = V.reshape(bsz * self.n_heads, src_len, self.d_head)

        # 5. Handle Mask Expansion
        if key_padding_mask is not None:
            # Mask is (bsz, src_len), needs to be (bsz * n_heads, src_len)
            # We repeat the mask for each head.
            # repeat_interleave is generally preferred here to keep batch alignment
            key_padding_mask = torch.repeat_interleave(key_padding_mask, self.n_hea

        # 6. Scaled Dot-Product Attention
        # Output Shape: (bsz * n_heads, tgt_len, d_head)
        attn_output = scaled_dot_product_attention(Q, K, V, key_padding_mask=key_pa

        # 7. Reshape and Transpose back
        # Split batch and heads: (bsz, n_heads, tgt_len, d_head)
        attn_output = attn_output.view(bsz, self.n_heads, tgt_len, self.d_head)

        # Transpose: (bsz, tgt_len, n_heads, d_head)
        attn_output = attn_output.transpose(1, 2)
```

Python

```
        # Flatten heads: (bsz, tgt_len, d_model)
        attn_output = attn_output.reshape(bsz, tgt_len, self.n_heads * self.d_head)

        # 8. Final Output Projection
        output = self.o_proj(attn_output)

        return output
        ############################################################################
```

Gemini one shots this part, nearly identical to the solution - fully vectorized. Difference is that it's using torch instead of einop. repeat. Also uses view + transpose + reshape instead of einops.rearrange, which is algebraically the same.

Next Step

The next section of this homework involves assembling the **TransformerEncoderLayer** and **TransformerDecoderLayer** using this multi-head attention block alongside feed-forward networks and layer normalization. Would you like to proceed with that?

part c:
Putting them together: Transformer

In this section, you will implement a Transformer encoder-decoder model for sequence-to-sequence...

Show thinking

Here is the implementation for the `make_positions` method within the `Transformer` class.

This method simply generates a sequence of indices `[0, 1, ..., seq_len-1]` and repeats it for every item in the batch. It mimics the standard behavior of learned positional embeddings where the absolute position index is what matters, regardless of the content or padding (padding is masked out later during the attention step).

Python

```python
def make_positions(self, input_ids, padding_mask):
    ################################################################################
    # TODO: implement this method
    ################################################################################
    # Get the sequence length from the input_ids shape (batch_size, seq_len)
    seq_len = input_ids.size(1)

    # Create a 1D tensor of positions: [0, 1, ..., seq_len - 1]
    # We ensure it uses the same device (CPU/GPU) and dtype (usually long) as :
    positions = torch.arange(seq_len, device=input_ids.device, dtype=input_ids.

    # Expand dimensions to match the batch size:
    # (seq_len) -> (1, seq_len) -> (batch_size, seq_len)
    # expand_as creates a view, which is memory efficient.
    return positions.unsqueeze(0).expand_as(input_ids)
    ################################################################################
```

Gemini one shots this. Ignores padding_mask like the solution, rows are [0, 1, ..., seq_len-1] replicated across batch which is what the solution does.

Overall, Gemini does a fantastic job of one shotting. It implements the std formula for softmax via padding and causal masking. Multihead Attention.forward does the same projections, head-splitting, masking, and reassembly as the official solutions. Only difference is style, behavior is equivalent. Positional indices (make_positions) generate abs positions for each sequence, matching the spec. Also, padding isn't baked into positions, just as reference in official solutions.

Next Step

This concludes the coding implementation for the Transformer model components! The next logical step in the assignment is usually to **instantiate the model, set up the** `Seq2SeqTrainer` **arguments, and run the training loop**. Would you like help configuring the training arguments for the XSum summarization task?

Executive Summary: Gemini produced a fully vectorized implementation that mirrored the official solutions. for both questions. Its solutions passed the provided test functions without added modifications. For handtransformer, gemini constructed valid K,Q,V and positional encodings, only differning in minor scaling constants such or whether queries depended on content or positional dimensions. For summarization, gemini's solutions were functionally identical to the official solutions, with differences limited to coding style (i.e. view, transpose, reshape instead of einops.rearrange). The attention mask handling, scaled by sqrt(d_head) and head aggregation all matched the reference logic. Overall, Gemini was impressive and efficiently coded the homework without hallucinating, and one shot everything.